

05/10/2020 - Procesos

1. Teoría básica + ejemplos
2. Ejercicios → Entrada a la práctica
→ Refuercen lo visto.
3. Práctica (En construcción)
4. Revision Lab.

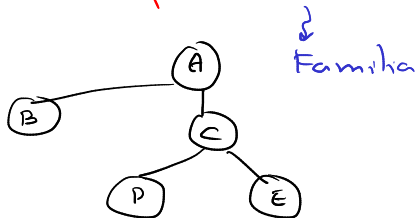
Desarrollo

1. Teoría y ejemplos:

Obtener información
getpid ✓
getppid

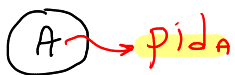
Gestionar
fork
exec [Familia] → Diagramas.
wait ✓

Jerarquía (Arbol)

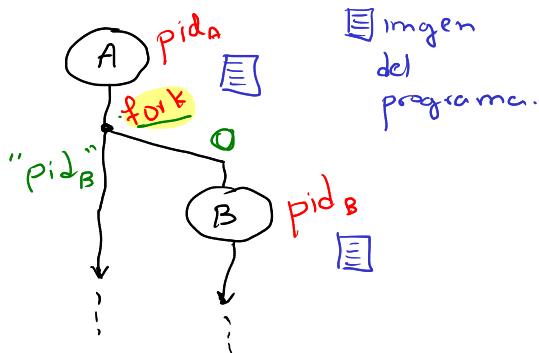


Funciones:

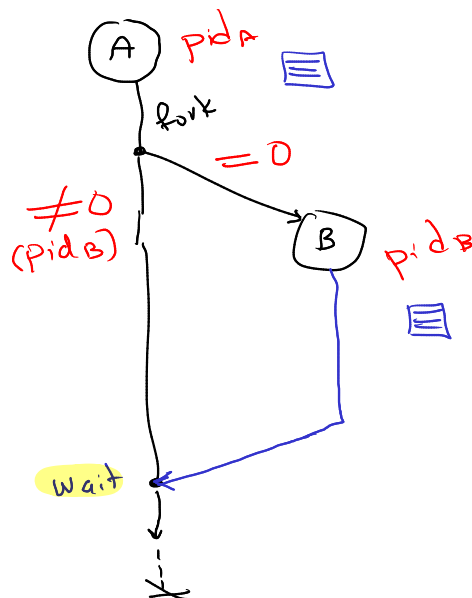
1. getpid ()



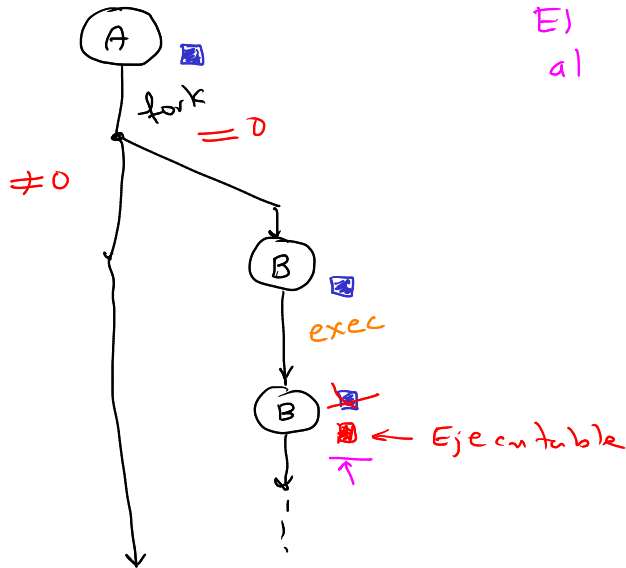
2. Fork



3. Wait



exec

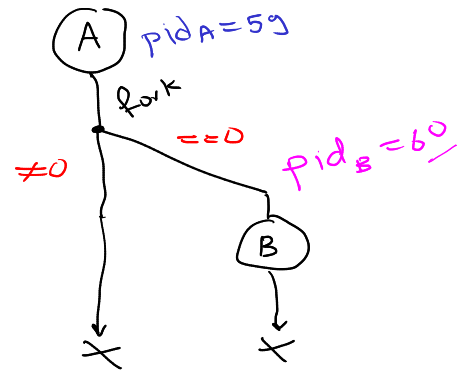


El código de A va ser diferente al de B

Ejemplo 1:

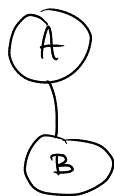
```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]) {
    printf("hello world (pid:%d)\n", (int) getpid());
    int rc = fork();
    if (rc < 0) {
        // fork failed; exit
        fprintf(stderr, "fork failed\n");
        exit(1);
    }
    else if (rc == 0) {
        // child (new process)
        printf("hello, I am child (pid:%d)\n", (int) getpid());
    }
    else {
        // parent goes down this path (original process)
        printf("hello, I am parent of %d (pid:%d)\n",
               rc, (int) getpid());
    }
    return 0;
}
```



```
> clang-7 -pthread -lm -o main main.c
> ./main
hello world (pid:59)
hello, I am parent of 60 (pid:59) ?
hello, I am child (pid:60)
> 
```

Jerarquía



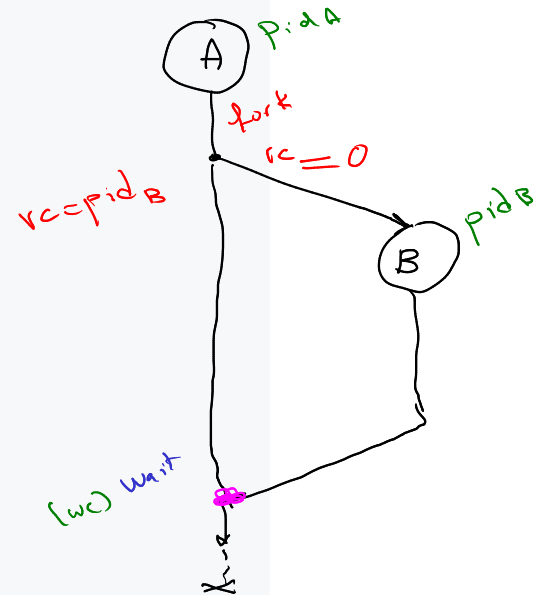
No se como va a ser la salida?
Indeterminada —

¿Como lograr una salida mas predecible.

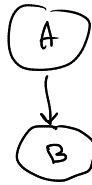
Ejemplo 2:

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
```

```
int main(int argc, char *argv[]) {
    ✓ printf("hello world (pid:%d)\n", (int) getpid());
    int rc = fork();
    if (rc < 0) {
        // fork failed; exit
        fprintf(stderr, "fork failed\n");
        exit(1);
    }
    else if (rc == 0) { ← hijo
        // child (new process)
        printf("hello, I am child (pid:%d)\n", (int) getpid());
        sleep(1);
    }
    else { ← pa
        // parent goes down this path (original process)
        int wc = wait(NULL); ←
        printf("hello, I am parent of (%d) (wc:%d) (pid:%d)\n", rc, wc, (int) getpid());
    }
    return 0;
}
```



Jerarquía



(B)

```
❯ clang-7 -pthread -lm -o main main.c
❯ ./main
hello world (pid:50)
hello, I am child (pid:51) *
hello, I am parent of 51 (wc:51) (pid:50)
❯
```

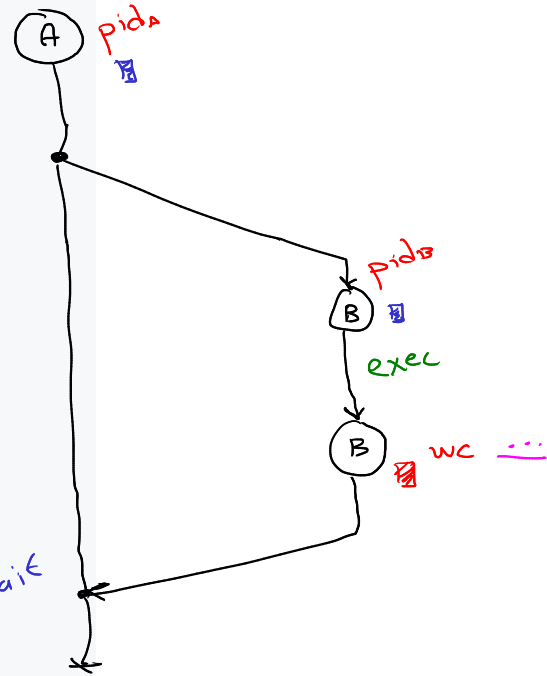
rc wc (wait) getpid()

Ejemplo 3.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>

int main(int argc, char *argv[]) {
    printf("hello world (pid:%d)\n", (int) getpid());
    int rc = fork();
    if (rc < 0) {
        // fork failed; exit
        fprintf(stderr, "fork failed\n");
        exit(1);
    } else if (rc == 0) {
        // child (new process)
        printf("hello, I am child (pid:%d)\n", (int) getpid());
        char *myargs[3];
        myargs[0] = strdup("wc"); // program: "wc" (word count)
        myargs[1] = strdup("p3.c"); // argument: file to count
        myargs[2] = NULL; // marks end of array
        execvp(myargs[0], myargs); // runs word count (wc p3.c)
        printf("this shouldn't print out");
    } else {
        // parent goes down this path (original process)
        int wc = wait(NULL);
        printf("hello, I am parent of %d (wc:%d) (pid:%d)\n",
              rc, wc, (int) getpid());
    }
    return 0;
}
```

glo
Recuerda
por
que
no voy



```
> clang-7 -pthread -lm -o main main.c
> ./main
hello world (pid:67)
hello, I am child (pid:68)
29 123 897 main.c
hello, I am parent of 68 (wc:68) (pid:67)
> wc main.c
29 123 897 main.c
> 
```

Jerarquía



Ejemplo 4.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <fcntl.h>
#include <assert.h>
#include <sys/wait.h>

int main(int argc, char *argv[]) {
    int rc = fork();
    if (rc < 0) {
        // fork failed; exit
        fprintf(stderr, "fork failed\n");
        exit(1);
    }
    else if (rc == 0) {

        // child: redirect standard output to a file
        close(STDOUT_FILENO);
        open("./p4.output", O_CREAT|O_WRONLY|O_TRUNC, S_IRWXU);
        // now exec "wc"...

        char *myargs[3];
        myargs[0] = strdup("wc"); // program: "wc" (word count)
        myargs[1] = strdup("p4.c"); // argument: file to count
        myargs[2] = NULL; // marks end of array
        execvp(myargs[0], myargs); // runs word count
    }
    else {
        // parent goes down this path (original process)
        int wc = wait(NULL);
        assert(wc >= 0);
    }
    return 0;
}
```

parte
adicional

→ Se cerró la salida estándar

→ Se abrió el
archivo al cual
redireccionar.

```
clang version 7.0.0-3-ubuntu0.18.04.1 (tags/RELEASE_700/final)
> wc main.c
33 114 850 main.c
> wc main.c > p4_2.output
> ls
main main.c p4_2.output p4.output
> cat p4_2.output
33 114 850 main.c
> ./main
> ls
main main.c p4_2.output p4.output
> cat p4.output
33 114 850 main.c
```

Ejemplo del comando ejecutado
en consola

Ejemplo redireccion

Visualizacion del archivo

Ejemplo ejecutando
el comando desde
un programa enc.

Visualizacion del archivo-